

Spring Boot Escrow Marketplace System - Version 2 (Financial Grade Design)

1. Financial Design Philosophy

Ledger-first architecture.
Booking system depends on financial system.
No balance mutation allowed.
All financial operations are append-only.

2. JPA Entity Design (Core Financial Models)

```
@Entity
@Table(name="accounts")
class Account {
    @Id UUID id;
    UUID ownerId;
    @Enumerated(EnumType.STRING)
    AccountType accountType;
    String currency;
    @Version Long version;
}

@Entity
@Table(name="transactions",
        uniqueConstraints=@UniqueConstraint(columnNames={"referenceId","referenceType"}))
class FinancialTransaction {
    @Id UUID id;
    String referenceId;
    String referenceType;
    @Enumerated(EnumType.STRING)
    TransactionStatus status;
    String idempotencyKey;
    Instant createdAt;
}

@Entity
@Table(name="ledger_entries")
class LedgerEntry {
    @Id UUID id;
    @ManyToOne FinancialTransaction transaction;
    @ManyToOne Account account;
    @Enumerated(EnumType.STRING)
    EntryType entryType; // DEBIT or CREDIT
    BigDecimal amount;
    Instant createdAt;
}
```

3. Core Transaction Posting Logic

```
@Transactional
public void postTransaction(List<PostingEntry> entries) {

    BigDecimal totalDebit = entries.stream()
        .filter(e -> e.type == DEBIT)
        .map(e -> e.amount)
        .reduce(BigDecimal.ZERO, BigDecimal::add);
```

```

BigDecimal totalCredit = entries.stream()
    .filter(e -> e.type == CREDIT)
    .map(e -> e.amount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);

if (!totalDebit.equals(totalCredit)) {
    throw new IllegalStateException("Debit/Credit mismatch");
}

FinancialTransaction txn = saveTransaction();

entries.forEach(e -> saveLedgerEntry(txn, e));
}

```

4. PostgreSQL DDL (Critical Constraints)

```

CREATE TABLE ledger_entries (
    id UUID PRIMARY KEY,
    transaction_id UUID NOT NULL REFERENCES transactions(id),
    account_id UUID NOT NULL REFERENCES accounts(id),
    entry_type VARCHAR(10) NOT NULL,
    amount NUMERIC(18,2) CHECK (amount > 0),
    created_at TIMESTAMP NOT NULL
);

ALTER TABLE transactions
ADD CONSTRAINT unique_reference UNIQUE(reference_id, reference_type);

-- Prevent deletion
REVOKE DELETE ON ledger_entries FROM public;

```

5. Escrow Sequence (Textual Diagram)

```

Renter → BookingService → PaymentService → Razorpay
Razorpay → WebhookController
WebhookController → FinancialService
FinancialService:
    DEBIT RENTER_CLEARING
    CREDIT ESCROW
Booking → PAID_LOCKED

```

6. Settlement Sequence (Scheduled Job)

```

Scheduler → SettlementService
SettlementService:
    Validate booking COMPLETED
    Check escrow balance
    Create transaction:
        DEBIT ESCROW
        CREDIT LENDER_WALLET
        CREDIT PLATFORM_REVENUE
    Update booking RELEASED

```

7. Withdrawal Flow with Concurrency Control

```
@Transactional
Account wallet = accountRepo.findByIdForUpdate(walletId);

BigDecimal balance = ledgerRepo.computeBalance(walletId);

if (balance.compareTo(withdrawAmount) < 0)
    throw new InsufficientFundsException();

Create transaction:
    DEBIT LENDER_WALLET
    CREDIT PLATFORM_BANK
```

8. Razorpay Webhook Hardening

- Verify HMAC signature
- Validate order amount
- Check idempotency key
- Store webhook event ID unique
- Wrap entire logic in @Transactional

9. Integration Test Scenarios

1. Double webhook delivery → must not duplicate ledger entries.
2. Concurrent withdrawals → must not allow negative balance.
3. Settlement job runs twice → must not double release funds.
4. Debit/Credit mismatch → transaction rejected.
5. Invalid booking state transition → rejected.

10. Advanced Extensions

- Reversal transactions (no delete)
- Reconciliation table for Razorpay payouts
- Outbox pattern for reliable event publishing
- Account statement API (ledger-based)
- Financial reporting queries